PROJECT PROGRESS REPORT

CVE-Guided Typestate Analysis for Recurring Vulnerability Detection in OpenSSL

Progress Meeting 1

11–16 October 2026

Meeting objective

Present the project scope, selected CVEs, current detection prototype, early validation evidence, evaluation approach, limitations, and the plan for the next milestone.

Project area:	Static analysis / software security
Target codebase:	OpenSSL
Status date:	16 October 2026
Team member 1:	Rutvij Wagh 230101113
Team member 2:	Udayraj Choudhary 230101107
Report type:	Progress meeting brief

Contents

1 Executive Summary **2**

2 Project Goal and Research Questions **2**
2.1 Goal . . . 2
2.2 Working research questions . . . 2
2.3 Typestate framing . . . 3

3 Work Completed: What We Did and How **3**
3.1 Step 1: Candidate discovery and selection . . . 3
3.1.1 What each criterion means . . . 3
3.1.2 Full score breakdown . . . 4
3.1.3 Why each candidate received its score . . . 4
3.1.4 Presentation-ready explanation of the selection . . . 5
3.2 Step 2: Official CVE record and patch-reference audit . . . 5
3.3 Step 3: Vulnerable-versus-fixed source analysis . . . 6
3.4 Step 4: Formal rule design . . . 6
3.5 Step 5: Checker implementation . . . 6
3.6 Step 6: Regression tests and runtime validation . . . 6
3.6.1 How the typestate rules were tested . . . 6
3.6.2 How we ran the tests on our device . . . 7
3.6.3 Runtime check with AddressSanitizer . . . 8
3.6.4 How the results were decided . . . 8
3.7 Step 7: Preliminary 16-case evaluation . . . 8
3.8 Summary: what the results establish . . . 9

4 Selected CVE Case Studies **9**
4.1 CVE-2023-0215: BIO chain lifetime . . . 9
4.2 CVE-2022-4450: PEM output-pointer cleanup . . . 10
4.3 CVE-2024-4741: SSL read-buffer lifecycle . . . 10

5 Recurrence Investigation **10**
5.1 How recurrence verification will be completed . . . 11

6 Reproducibility and How the Work Was Run **11**
6.1 Snippet evaluation . . . 12
6.2 Existing CVE regression tests . . . 12

7 Limitations and Risks **12**

8 Conclusion **13**

References and Evidence Sources **13**

1 Executive Summary

The project investigates whether recurring OpenSSL API and resource-lifecycle mistakes can be expressed as typestate-like rules and detected by targeted static checks. The initial research foundation and prototype are in place. We created a candidate register of eight CVEs, selected three primary cases, compared vulnerable and fixed implementations, specified three detection rules, implemented a Python checker, and prepared regression and snippet-level evaluation tests.

The strongest runtime evidence currently documented is for CVE-2023-0215: an AddressSanitizer (ASan) run on the vulnerable revision reported a heap-use-after-free. The recorded fixed-revision run did not produce the corresponding ASan report. For CVE-2022-4450 and CVE-2024-4741, the current evidence is source-pattern regression testing rather than equivalent runtime reproductions.

The packaged evaluation harness contains 16 labelled snippets. The included evaluation script was rerun while preparing this meeting report: it exited with status 0 and reported 3 true positives, 13 true negatives, 0 false positives, and 0 false negatives on that suite. This is useful prototype evidence, but the cases are largely hand-written/synthetic snippets and do not establish general detection accuracy on OpenSSL source.

Area	Current position
Scope and candidate shortlist	Eight candidates registered; three primary cases selected.
Rule implementation	Three CVE-specific source-pattern heuristics in Python.
Historical validation	Vulnerable/fixed source-pattern checks for all three selected cases.
Runtime evidence	ASan evidence recorded for CVE-2023-0215 only.
Evaluation harness	16 snippet cases pass; results are limited to this small labelled suite.
Recurrence investigation	Ten candidate locations are recorded; several still need pinned-source excerpts and line-level validation.
Final readiness	Part 1 has substantial groundwork, but broader evaluation and fully reproducible source-level audit are still outstanding.

2 Project Goal and Research Questions

2.1 Goal

To investigate whether selected OpenSSL vulnerabilities caused by unsafe API/resource lifecycle handling can be represented as typestate-like invariants and whether targeted source checks can detect the same classes of mistakes in related code.

2.2 Working research questions

1. Can the root causes of selected OpenSSL CVEs be described as violations of object state, ownership, or lifecycle invariants?

2. Can those invariants be encoded as simple static source-pattern rules that distinguish the selected vulnerable versions from their corresponding fixed versions?

3. Do related OpenSSL code locations exhibit similar patterns, and how can plausible recurrence candidates be triaged without incorrectly claiming new vulnerabilities?

4. What additional evaluation is needed to assess false positives, false negatives, and generalizability beyond the cases used to write the rules?

2.3 Typestate framing

Typestate describes how legal operations depend on an object's current state. For each resource, the analysis records: (i) the object or resource, (ii) its abstract states, (iii) allowed transitions, (iv) forbidden transitions, (v) evidence in the implementation, and (vi) a proposed detection rule. In the current prototype, this is a design model for the rules, not a complete compiler-level typestate engine.

3 Work Completed: What We Did and How

3.1 Step 1: Candidate discovery and selection

We created `cve_candidates_openssl_v2.csv` with eight CVE candidates. The register records a short description, affected-version scope, relevant code location, official advisory/record URL, patch reference, lifecycle or API hypothesis, selection status, rationale, and planned tests. We then applied a five-criterion rubric. Each criterion received an integer score from 0 to 2, and all five criteria had equal weight, so the maximum score was $5 \times 2 = 10$.

3.1.1 What each criterion means

The scores are qualitative project-prioritization judgments. They are not severity scores, probabilities, or measured detection results. We assigned them using the state of our evidence during candidate screening, and then checked the selected cases in source revisions.

Criterion	0 points	1 point	2 points
Patch/source availability	No reliable advisory, relevant source, or fixing change identified.	An advisory or likely source location exists, but the exact patch, branch, or revision still needs substantial confirmation.	Relevant source and fixing change/revisions can be identified and compared.
Root-cause clarity	The likely programming mistake is unclear from the available evidence.	The cause is plausible, but the interpretation or exact failing path remains uncertain.	Source and patch give a concrete, explainable account of the defect and correction.
Typestate/API rule fit	The issue does not clearly involve an object state, API contract, ownership, resource lifetime, or cleanup invariant.	A reusable state/API rule is possible, but the issue may mainly be parameter validation or a narrow check.	The issue strongly supports an invariant with meaningful states, transitions, ownership, or lifecycle requirements.
Recurrence potential	No plausible related pattern or reusable contract has been identified.	Similar code may exist, but recurrence is uncertain or narrow.	The same type of contract violation could plausibly occur at multiple related API or cleanup sites. This is potential, not proof of recurrence.
Time feasibility	The source/build/test work is not realistic within the project schedule.	Significant version, patch, build, or reproduction uncertainty remains.	The case is bounded enough to inspect, formulate a rule for, and test within the current milestone.

3.1.2 Full score breakdown

CVE	Patch	Cause	Rule fit	Recurrence	Time	Total	Decision
CVE-2023-0215	2	2	2	2	2	**10**	Selected
CVE-2022-4450	2	2	2	2	2	**10**	Selected
CVE-2024-4741	2	2	2	1	2	**9**	Selected
CVE-2023-5363	2	2	1	1	2	**8**	Not selected
CVE-2023-0401	2	2	1	1	2	**8**	Not selected
CVE-2023-0216	2	2	1	1	2	**8**	Not selected
CVE-2026-45447	1	2	2	1	1	**7**	Follow-up candidate
CVE-2026-35188	1	1	2	1	1	**6**	Follow-up candidate

The five numeric columns correspond, in order, to patch/source availability, root-cause clarity, typestate/API rule fit, recurrence potential, and time feasibility. The score is the sum of those five values. The selection was based on a combination of a high total and a good fit to this project's research question, not on vulnerability severity alone.

Scoring-rationale note. The component scores are taken from the project's recorded shortlist and the CVE-2023-0216 total has been reconciled to the arithmetic sum of its components. The candidate-by-candidate explanations below are a clarified reconstruction of why those recorded components fit the project criteria, using the screening notes and current source analysis; they should not be read as verbatim quotations from an original scoring meeting. If the original team notes contain a more specific rationale, keep those as the primary record.

3.1.3 Why each candidate received its score

- **CVE-2023-0215 (2,2,2,2,2 = 10).** The relevant vulnerable/fixed source revisions and fixing commit were identified for the project. The patch exposed the BIO-chain cleanup problem, and the invariant can be stated as restoring/detaching the chain before freeing the relevant BIO. Ownership and lifetime patterns are plausible to investigate in related BIO paths. The case was feasible within the milestone and later provided an AddressSanitizer reproduction on the vulnerable revision.

- **CVE-2022-4450 (2,2,2,2,2 = 10).** The source and fix were identifiable, and the key change makes the failure-cleanup contract easy to explain: after freeing the header/data buffers, the caller-visible output pointers are reset. That is a strong pointer-state and cleanup rule, with plausible analogues in APIs that return allocated output through pointer parameters. The affected function was sufficiently bounded for a first rule and vulnerable/fixed test.

- **CVE-2024-4741 (2,2,2,1,2 = 9).** The affected buffer-release code and fix were identifiable, and the issue maps well to resource lifetime and packet-state consistency. Recurrence received 1 rather than 2 because the relevant conditions are more specific to SSL read-buffer handling, so broader reuse of the exact rule was less certain at screening time. The source comparison and a first targeted check were feasible.

- **CVE-2023-5363 (2,2,1,1,2 = 8).** The patch/source and broad cause were considered identifiable, but the problem centres on cipher key/IV length handling. That may be better characterized as parameter/configuration validation than a resource-lifecycle typestate rule, so both rule fit and recurrence potential were scored 1. It was feasible to investigate, but less aligned with our initial rule design.

- **CVE-2023-0401 (2,2,1,1,2 = 8).** The broad NULL-dereference cause and source/fix were considered clear enough for screening. However, a missing NULL check does not automatically establish a meaningful reusable state-transition invariant; its recurrence as the same typestate mistake was uncertain. It remained a contrast candidate rather than a primary case.
- **CVE-2023-0216 (2,2,1,1,2 = 8).** The pointer-dereference issue and source/fix were considered identifiable, but the case appeared more tied to a particular PKCS#7 decoding path than to a strong, general resource-lifecycle contract. Its fit and recurrence scores were therefore 1. The total is 8 because the five component scores sum to $2+2+1+1+2=8$; an earlier draft showed 7, and that arithmetic inconsistency was reconciled here.
- **CVE-2026-45447 (1,2,2,1,1 = 7).** The reported use-after-free in `PKCS7_verify()` looked strongly related to object lifetime and caller-owned BIO handling, hence the 2 for cause and rule fit. At the time of screening, the exact project-level branch/revision comparison and test path had not been established, so patch/source and time feasibility received 1. The relationship to our BIO rule still needed independent checking, so recurrence was 1.
- **CVE-2026-35188 (1,1,2,1,1 = 6).** The reported OCSP-response double-free was a plausible ownership/cleanup-rule candidate, which supports a 2 for rule fit. However, the exact source-level failure path and patch mapping were not yet established in the project at screening time, so patch/source and cause were scored 1. Recurrence remained uncertain and the additional source/version work reduced time feasibility.

The 2026 candidates' lower patch/source or feasibility scores reflect the amount of project-specific source and test work outstanding at the time of screening; they do not mean the advisories were necessarily invalid. Likewise, a recurrence score of 2 means that recurrence seemed plausible enough to investigate, not that another occurrence had already been confirmed.

The three selected cases offer complementary but related patterns: (i) BIO-chain lifetime and cleanup, (ii) caller-visible output-pointer state after PEM cleanup, and (iii) TLS read-buffer and dependent packet state. This gave us three concrete cases for a first pipeline while keeping the scope manageable.

3.1.4 Presentation-ready explanation of the selection

We used five equally weighted criteria because a useful initial research case needed more than high severity: it needed a traceable fix, an explainable root cause, a meaningful state/API invariant, a plausible route to investigating recurrence, and a realistic amount of work. CVE-2023-0215 and CVE-2022-4450 each scored 10/10, while CVE-2024-4741 scored 9/10. These three were selected because their rules address complementary resource-management failures but share a common theme of keeping object lifetime, pointer state, and ownership consistent. The score is an explicit rationale for prioritization, not proof that the chosen cases are the only suitable ones or that recurrence has already been demonstrated.

3.2 Step 2: Official CVE record and patch-reference audit

We prepared `official_cve_verification.md` and updated the candidate CSV to distinguish official metadata checks from project-level reproductions. The audit corrected affected-version descriptions that were inaccurate in an earlier draft, particularly for CVE-2023-0401, CVE-2023-0216, CVE-2026-45447, and CVE-2026-35188. We linked the official OpenSSL vulnerability index, relevant release/advisory pages, and patch commits.

This verification stage means the advisory metadata and patch references were checked against official listings. It does *not* mean that all eight CVEs were independently built and reproduced by this project. For non-selected candidates, exact source paths and branch-specific patch context should still be verified before claiming source-level reproduction or using those cases as evaluation examples.

3.3 Step 3: Vulnerable-versus-fixed source analysis

For each selected CVE, we identified a vulnerable revision and a fixing revision, compared the relevant cleanup or state-handling code, and wrote a root-cause explanation. The cases and main invariants are summarized in Section 4. This source comparison informed the rules rather than relying only on vulnerability titles or advisory summaries.

3.4 Step 4: Formal rule design

We wrote `typestate_rules_spec_v2.md` with the tracked resource, abstract states, allowed transitions, forbidden transitions, failure mode, detection logic, and limitations for each rule:

- **TS-BIO-01**: BIO chain lifetime and detach-before-free ownership contract.
- **TS-PEM-02**: reset caller-visible output pointers after internal cleanup on a failure path.
- **TS-SSL-03**: prevent freeing the TLS read buffer while dependent record-layer state still refers to it, and keep dependent state consistent.

3.5 Step 5: Checker implementation

The current Python checker is a deliberately narrow prototype. It uses function extraction and string-pattern checks to identify three known patterns. It does not provide full abstract interpretation, path-sensitive control-flow analysis, interprocedural ownership tracking, alias analysis, or symbolic execution. The rule specification is broader than the actual heuristics in places, especially for TS-SSL-03; this difference is explicitly treated as a limitation rather than claiming the implementation enforces the full invariant.

3.6 Step 6: Regression tests and runtime validation

We used a layered test strategy, because a source-pattern check, a vulnerable/fixed regression pair, a runtime sanitizer experiment, and a synthetic benchmark answer different questions. We did not treat all of them as equivalent evidence.

3.6.1 How the typestate rules were tested

For every selected CVE, the fixing patch was first used to state an expected invariant. We then applied the checker to the historical vulnerable and fixed source files. The expected result was:

- **Vulnerable source:** the rule should print `ALERT`, because the source contains the target pattern that violates the inferred invariant.
- **Fixed source:** the rule should print `NO ALERT`, because the specific defect pattern addressed by the patch should no longer be present.
- **Pair marked PASS:** both outputs match those expectations. A pair is not marked PASS just because one side has an alert.

This is a regression oracle based on the known vulnerable/fixed patch pair. It tests whether the implemented heuristic distinguishes those selected versions; it does not prove that every possible unsafe program will be detected or that every alert in a different file is a real vulnerability.

Case	Vulnerable src.	Fixed src.	Pair result	Evidence level
CVE-2023-0215	ALERT	NO ALERT	**PASS**	Source-pattern regression plus recorded ASan runtime evidence.
CVE-2022-4450	ALERT	NO ALERT	**PASS**	Source-pattern regression only; no runtime reproduction claimed.
CVE-2024-4741	ALERT	NO ALERT	**PASS**	Source-pattern regression only; no runtime reproduction claimed.

3.6.2 How we ran the tests on our device

The terminal prompt recorded during this project was similar to:

```
[OSS CAD Suite] rutvij@Rutvij:/mnt/d/CVE-Typestate-Project$
```

This shows that the project was accessed from a Linux/WSL-style shell and that the Windows `D:` drive was mounted at `/mnt/d`. The project root used in the recorded commands was `/mnt/d/CVE-Typestate-Project`, corresponding to `D:\CVE-Typestate-Project` in Windows File Explorer. The `OSS CAD Suite` text is an environment/toolchain label in the prompt, not the name of the project directory. The evidence available for this report does not record the exact WSL distribution version, compiler version, or output of `python3 -version`; those environment details should be captured for a fully reproducible final submission.

The project-level checks were run from the root directory. The following commands describe the recorded test workflow; re-run them before a final demonstration and retain the actual terminal output:

Listing 1: Syntax checks and source-pattern regression tests

```
cd /mnt/d/CVE-Typestate-Project
python3 -m py_compile typestate_check.py
bash -n tests/test_all_cves.sh
bash tests/test_cve_2023_0215.sh
bash tests/test_cve_2022_4450.sh
bash tests/test_all_cves.sh
```

The syntax checks verify that the Python file can be compiled and the combined shell script can be parsed. The CVE scripts then run the checker on the selected source pairs. The recorded combined run reported an alert for each vulnerable source and no alert for each corresponding fixed source, and concluded: `PASS: all three vulnerable/fixed pairs were distinguished.`

For a future fully reproducible run, capture the versions and exact worktree revisions as well. These are recommended metadata commands; their output was not preserved in the evidence supplied for this report:

Listing 2: Environment metadata to record on the device

```
uname -a
python3 --version
gcc --version
git -C /mnt/d/CVE-Typestate-Project/openssl-cve-2023-0215-vulnerable rev-parse HEAD
git -C /mnt/d/CVE-Typestate-Project/openssl-cve-2023-0215-fixed rev-parse HEAD
```

3.6.3 Runtime check with AddressSanitizer

CVE-2023-0215 was also investigated at runtime in the appropriate vulnerable and fixed OpenSSL worktrees. From the root of the corresponding built worktree, the recorded command was:

Listing 3: Recorded CLI command for CVE-2023-0215

```
./apps/openssl cms -encrypt -in test/smcont.txt -stream \
  -recip test/smime-certs/smec1.pem -des
```

On the vulnerable checkout, AddressSanitizer reported a heap-use-after-free. The noted use site was `BIO_pop()` in `crypto/bio/bio_lib.c:678`; the relevant freed object was associated with cleanup in `BIO_new_NDEF()` in `crypto/asn1/bio_ndef.c:100`. The trigger exercised CMS streaming encryption with `-des`, where cipher initialization failed and the error-cleanup path was followed.

The same command was run on the fixed checkout and did not produce the corresponding ASan memory-safety report. However, the command still returned an error. Therefore, the result is described narrowly: the specific memory-safety report was not reproduced in that fixed-version run. We do not claim that this command completed a successful CMS encryption operation. Equivalent runtime reproductions for CVE-2022-4450 and CVE-2024-4741 have not been recorded.

3.6.4 How the results were decided

The project kept three levels of evidence separate:

1. **Source-pair regression result.** Compare actual checker output with the expected labels justified by the known vulnerable/fixed source pair. Mark the pair PASS only when vulnerable=ALERT and fixed=NO ALERT.

2. **Runtime sanitizer result.** Check whether the vulnerable execution emits the targeted memory-safety diagnostic and whether the corresponding diagnostic appears in the fixed run. Preserve command failures and other output as well; absence of an ASan report in one run is not a proof of safety.

3. **Synthetic benchmark result.** Compare the checker output with the hand-assigned expected labels for the 16 code snippets. This is a unit-test-style sanity check because the cases were written around the implemented heuristics; it is not an independent estimate of real-world performance.

This separation matters because a test can pass its stated oracle while the oracle itself remains narrow. In particular, the three vulnerable/fixed source-pair tests are meaningful evidence that the current checker recognizes the patterns used to construct those rules. They are not an independent evaluation of whether the same rules generalize to other OpenSSL functions.

3.7 Step 7: Preliminary 16-case evaluation

The package includes `typestate_checker_eval_v2.py`, a standard-library-only harness containing 16 labelled code snippets: three vulnerable snippets expected to alert, three corresponding fixed snippets, eight safe-control snippets, and two non-typestate controls. The evaluation script was rerun while preparing the report and returned exit status 0.

Measure	Observed result on this 16-case suite
True positives (TP)	3
True negatives (TN)	13
False positives (FP)	0
False negatives (FN)	0
Total	16
Precision	100%
Recall	100%
Specificity	100%
F1 score	100%
Observed accuracy on these labels	100% (16/16)

The reported metrics are obtained from the suite's expected labels and actual outputs: precision is $TP/(TP+FP) = 3/(3+0)$; recall is $TP/(TP+FN) = 3/(3+0)$; specificity is $TN/(TN+FP) = 13/(13+0)$; and accuracy on this suite is $(TP+TN)/16 = (3+13)/16$. Because the test examples are synthetic, small, and closely tied to the patterns encoded in the rules, these 100% figures do *not* estimate performance on unseen OpenSSL code.

The harness analyzes snippets embedded in the Python test script. It does not load each corresponding OpenSSL source file from the worktrees. This means the benchmark demonstrates that the current implementation behaves as expected on its authored examples, but does not independently validate the checker against the real program or establish generalization. A stronger future benchmark must contain independently selected source locations, harder near-miss and adversarial cases, and expected labels reviewed separately from the rule implementation. The output log used in the report is `eval_v2_stdout.log`; it should be regenerated from a fresh run rather than manually edited.

3.8 Summary: what the results establish

- **Established for this milestone:** the project has a documented candidate-scoring process, source comparisons for three chosen CVEs, three implemented CVE-specific heuristics, and a combined test run that distinguished each selected vulnerable/fixed source pair.
- **Runtime evidence:** CVE-2023-0215 has a recorded vulnerable-version ASan heap-use-after-free and a fixed-version run without the corresponding report, subject to the command-failure caveat described above.
- **Preliminary only:** the 16-case synthetic benchmark achieved all expected labels, but does not establish generalized accuracy.
- **Not established:** comprehensive typestate analysis, broad precision/recall on real OpenSSL code, runtime reproduction of all three CVEs, or discovery of a new recurring vulnerability.

4 Selected CVE Case Studies

4.1 CVE-2023-0215: BIO chain lifetime

Root cause. The affected `BIO_new_NDEF()` error handling could free a filter BIO before correctly restoring the caller-owned BIO chain links. A later operation on the BIO chain could follow a stale reference, causing a use-after-free.

Typestate invariant. A filter BIO linked into a caller-owned chain must be detached or the chain must otherwise be restored before the filter is freed. The simplified states are `ALLOCATED` →

`LINKED_IN_CHAIN` → `DETACHED` → `FREED`. A forbidden transition is `LINKED_IN_CHAIN` → `FREED` while stale chain references remain.

Rule. TS-BIO-01 inspects the pattern of BIO linking and error cleanup, looking for a release without an equivalent detach/chain-restoration step. The current implementation approximates this with source-pattern matching.

Evidence. Vulnerable/fixed rule tests exist. The vulnerable revision also has documented ASan heap-use-after-free evidence; the fixed revision did not produce the corresponding report in the recorded run. This is the strongest runtime evidence in the project so far. It is not evidence that the checker has found a new vulnerability.

4.2 CVE-2022-4450: PEM output-pointer cleanup

Root cause. A failure path in `PEM_read_bio_ex()` freed internal header/data buffers but could leave caller-visible output pointers referring to the released memory. A caller attempting cleanup after failure could free the same storage again.

Typestate invariant. An internally freed output buffer must not remain reachable through a caller-visible output pointer when the function returns failure. The modeled transition is `ALLOCATED_INTERNAL` → `FREED_ON_ERROR` → `CLEARED_TO_NULL`.

Rule and evidence. TS-PEM-02 checks for freeing output buffers and corresponding pointer resets in the cleanup path. The current evidence is source-pattern regression testing, not a runtime reproduction.

4.3 CVE-2024-4741: SSL read-buffer lifecycle

Root cause. The `SSL_free_buffers()` path could release the read buffer while record-layer data still depended on it in identified scenarios. Later use of that state could access freed memory. The upstream correction guards buffer release according to record-layer state; branch-specific patches should be used for a final source-level comparison.

Typestate invariant. A buffer cannot transition to `FREED` while current or unread record data still depends on it. Its associated packet pointer and length must also be kept consistent with the buffer's lifetime.

Rule and evidence. TS-SSL-03 checks a narrow pointer/length reset pattern. It does not implement the complete upstream guard and should be described as a heuristic approximation. The packaged result is source-pattern regression evidence; runtime reproduction is not claimed.

5 Recurrence Investigation

The file `recurrence_investigation_v2.md` records ten candidate locations in BIO, PEM, SSL record, CMS, PKCS#12, EVP, and PKCS#7 code. The current evidence is mixed: some rows refer to earlier manual notes, while others use illustrative snippets in the evaluation harness. The package does not contain pinned source commits and line-numbered excerpts for every row. Therefore, the current recurrence result is a **candidate search log**, not a completed source audit and not a confirmed discovery of a new vulnerability.

ID	Candidate location recorded	Current interpretation
R01	`crypto/asn1/asn_mime.c` – `B64_write_ASN1`	Safe-control candidate; inspect exact source revision and lines.
R02	`crypto/encode_decode/decoder_lib.c` – `OSSL_DECODER_from_bio`	Safe-control candidate; verify real cleanup branch.

ID	Candidate location recorded	Current interpretation
R03	`crypto/bio/bss_acpt.c` – `acpt_state`	Related ownership pattern; likely outside the exact caller-owned filter-chain invariant.
R04	`crypto/pem/pem_lib.c` `PEM_X509_INFO_read_bio`	– Weak match to TS-PEM-02; do not count as a direct test without validating output-pointer semantics.
R05	`ssl/record/rec_layer_s3.c` `ssl3_setup_read_buffer`	– Safe-control candidate; initialization alone does not establish release safety.
R06	`crypto/cms/cms_smime.c` – `CMS_dataInit`	Safe-control candidate; verify real cleanup path.
R07	`crypto/pkcs12/p12_decr.c` `PKCS12_item_decrypt_d2i`	– Safe-control candidate; current evidence is illustrative.
R08	`crypto/cms/cms_enc.c` `cms_EncryptedContent_init_bio`	– Safe-control candidate; verify actual source and ownership.
R09	`crypto/evp/evp_enc.c` `EVP_EncryptInit_ex2`	– Non-typestate contrast, not evidence about BIO or PEM lifecycle.
R10	`crypto/pkcs7/pk7_asn1.c` – `d2i_PKCS7`	Non-typestate/pointer-validation contrast; not a validated reproduction of CVE-2023-0216.

A useful follow-up is CVE-2026-45447, whose official record describes a use-after-free in `PKCS7_verify()` involving a caller-owned BIO. Its ownership aspect makes it relevant to TS-BIO-01, but the project has not yet run this candidate through the checker or reproduced it. It is therefore a follow-up hypothesis, not a detected recurrence result.

5.1 How recurrence verification will be completed

For each candidate row, the next source-level audit should preserve:

1. the exact upstream commit hash (for example, from `git rev-parse HEAD`);
2. the precise source location and matching line numbers (for example, using `git grep -n` and `nl -ba`);
3. a short excerpt showing the state/ownership operation and its cleanup path;
4. the expected invariant and the reason the example is safe, unsafe, or unresolved;
5. a conclusion label such as safe control, rejected candidate, possible recurrence, or unresolved.

No new vulnerability is claimed in this meeting report.

6 Reproducibility and How the Work Was Run

6.1 Snippet evaluation

The bundled evaluation harness is standard-library-only and does not need a local OpenSSL build. From the extracted Version 3 package directory, run:

Listing 4: Re-run the packaged snippet-level evaluation

```
set -o pipefail
python3 --version
python3 typestate_checker_eval_v2.py | tee eval_v2_stdout.log
```

The run used while preparing this report returned exit status 0. The resulting output reports all 16 cases passing and the confusion-matrix counts shown in Section 4. The output log can be regenerated with this command; do not edit the log by hand to make it match expected results.

6.2 Existing CVE regression tests

The broader project workspace contains targeted tests such as `tests/test_cve_2023_0215.sh` and `tests/test_cve_2022_4450.sh`, plus worktrees for vulnerable/fixed revisions. These worktrees and all ASan build artifacts are not included in the small Version 3 ZIP. Consequently, the snippet evaluation is reproducible from that ZIP alone, but the runtime ASan result requires the separate project workspace, correct revisions, and saved build/test evidence.

The documented CVE-2023-0215 reproduction command was:

Listing 5: Recorded OpenSSL CLI command for the ASan reproduction

```
./apps/openssl cms -encrypt -in test/smcont.txt -stream \
  -recip test/smime-certs/smec1.pem -des
```

Use this command only in the project's controlled vulnerable/fixed OpenSSL test worktrees. Preserve the full console output and build configuration. The report does not claim that this CLI operation completed successfully in the fixed build; the claim is specifically about the corresponding ASan report not appearing in the recorded fixed-version run.

7 Limitations and Risks

- **Narrow analyzer:** the checker is composed of CVE-specific string-pattern heuristics, not a generalized typestate analysis engine.
- **Benchmark design:** all 16 current examples are snippet-based and the rule patterns were designed around the selected CVEs. Perfect results on this suite are not evidence of broad generalization.
- **Limited runtime validation:** only CVE-2023-0215 has the documented ASan evidence. The other two selected cases have source-pattern test results.
- **Recurrence evidence:** the ten-row candidate register is not yet a line-by-line source audit against pinned revisions.
- **Reproduction boundary:** the Version 3 ZIP reproduces the Python snippet evaluation; it does not contain all OpenSSL worktrees/build artifacts needed to recreate ASan from scratch.
- **No new bug claim:** no recurrence candidate is currently reported as a newly confirmed vulnerability.

8 Conclusion

The project has progressed beyond initial topic selection: a candidate register, shortlist, selected-CVE analyses, three targeted detection rules, initial vulnerable/fixed tests, one documented ASan reproduction, a ten-row recurrence candidate log, and a 16-case snippet evaluation are in place. The current prototype is a credible foundation for the next stage, but it is not a general typestate analyzer and the recurrence audit is not yet complete. The next milestone should focus on source-pinned evidence, a stronger evaluation set, and repeatable execution rather than rebuilding the existing foundation.

References and Evidence Sources

[1] OpenSSL, *Vulnerabilities index*. Official vulnerability descriptions and affected-release information. https://www.openssl-library.org/news/vulnerabilities/

[2] OpenSSL, *OpenSSL 3.0 release notes*, including the 3.0.8 security-fix summary for CVE-2023-0215, CVE-2023-0216, CVE-2023-0401, and CVE-2022-4450. https://openssl-library.org/news/openssl-3.0-notes/

[3] OpenSSL, *Vulnerabilities 3.0*, including CVE-2023-5363. https://www.openssl-library.org/news/vulnerabilities-3.0/

[4] OpenSSL, *Vulnerabilities 3.3*, including CVE-2024-4741 and the affected branch list. https://www.openssl-library.org/news/vulnerabilities-3.3/

[5] OpenSSL source repository, fix commit for CVE-2023-0215. https://github.com/openssl/openssl/commit/8818064ce3c3c0f1b740a5aaba2a987e75bfbafd

[6] OpenSSL source repository, fix commit reference for CVE-2022-4450. https://github.com/openssl/openssl/commit/63bcf189be73a9cc1264059bed6f57974be74a83

[7] OpenSSL source repository, 3.3 fix commit reference for CVE-2024-4741. https://github.com/openssl/openssl/commit/e5093133c35ca82874ad83697af76f4b0f7e3bd8

[8] OpenSSL, CVE-2026-45447 official listing; NVD record. https://www.openssl-library.org/news/vulnerabilities/ and https://nvd.nist.gov/vuln/detail/CVE-2026-45447.

[9] OpenSSL, CVE-2026-35188 official listing; NVD record. https://www.openssl-library.org/news/vulnerabilities/ and https://nvd.nist.gov/vuln/detail/CVE-2026-35188.

Evidence packages consulted: Version 3 ZIP and its files `OpenSSL_Typestate_Analysis_Final_Report_v2.md`, `official_cve_verification.md`, `recurrence_investigation_v2.md`, `typestate_rules_spec_v2.md`, `typestate_checker_eval_v2.py`, `eval_v2_stdout.log`, `README_v2.md`, and `cve_candidates_openssl_v2.csv`. The evaluation script was re-executed during preparation of this report.